

Leveraging large language models for automating water distribution network optimization

Jian Wang^a, Guangtao Fu^{a,*}, Dragan Savic^{a,b,*}

^a Centre for Water Systems, University of Exeter, Exeter EX4 4QF, UK

^b KWR Water Research Institute, Nieuwegein 3430 BB, the Netherlands

ARTICLE INFO

Keywords:

Water distribution network
Large language model
AI agent
DeepSeek
Hydraulic optimization

ABSTRACT

Effective management of Water Distribution Networks (WDNs) is essential to ensure efficient and reliable water supply in cities. However, many management tasks require complex system modelling and optimization approaches, which heavily rely on specialized domain expertise and human resources. Recent advancements in Large Language Models (LLMs) offer promising opportunities to automate complex hydraulic decision-making tasks. This study presents an LLM-based agent framework to automate WDN management tasks. Two tasks are considered to evaluate the feasibility and limitations of LLM agents: hydraulic model calibration and pump operation optimization. The key component of the proposed framework is an Orchestrating Agent that interprets tasks and system states, generates update strategies or executable code, and interacts with three specialized agents to carry out implementation: a Knowledge Agent performing reasoning based on hydraulic principles, a Modelling Agent that interfaces with hydraulic simulation tool EPANET, and a Coding Agent that executes code and returns output feedback. To assess the capabilities of these agents, the framework was systematically tested on two benchmark WDNs - Net2 and Anytown. The results indicate that the reasoning capability demonstrated through interaction with the Knowledge Agent effectively replicates expert-level hydraulic thinking, though it lacks numerical precision. In contrast, the Modelling Agent, which integrates external simulation tools, enhances reliability, although interpreting and enforcing numerical constraints expressed in natural language remain challenging, particularly in looped networks such as Anytown where the agent often converged to suboptimal solutions. Furthermore, the Coding Agent, where code for optimization algorithms is iteratively generated and executed, delivers the most consistent and accurate performance across both networks, underscoring its practical potential. These findings highlight the potential of LLM-based agents for automated, accurate hydraulic optimization, and represent a significant step toward LLM-driven multi-agent frameworks for hydraulic decision-making. This work establishes a foundation for future advancements in specialized, domain-focused LLM applications in complex hydraulic management scenarios.

1. Introduction

Water Distribution Networks (WDNs) play a critical role in ensuring the continuous delivery of safe, reliable, and high-quality water to consumers (Fu et al., 2022a; Sarbu and Popa-Albu, 2023; Zaman et al., 2021). Their operational priorities span a wide range of objectives, including 24/7 service continuity, sustainable energy and operational costs, minimal water losses, limited environmental and social impact, and satisfactory customer service (Sharif et al., 2022; Zarei et al., 2022). However, meeting these objectives has become increasingly difficult in a rapidly changing context. Many WDNs face the challenges of ageing

infrastructure, evolving demand patterns, reduced workforce continuity, and the growing frequency of extreme weather events (Gong et al., 2023; Sela et al., 2025). In this complex environment, hydraulic optimization tasks are fundamental to improving system efficiency, resilience, and sustainability (Nedaei, 2025).

Among these hydraulic optimization tasks, hydraulic model calibration and pump operation optimization are two critical and extensively studied problems, due to their significant impacts on network performance and operational costs (Batista do Egito et al., 2023; Makaremi et al., 2017; Zhang et al., 2018). The two problems are also specific examples of the two general types of decision-making

* Corresponding authors.

E-mail addresses: g.fu@exeter.ac.uk (G. Fu), d.savic@exeter.ac.uk (D. Savic).

<https://doi.org/10.1016/j.watres.2025.124536>

Received 17 June 2025; Received in revised form 28 August 2025; Accepted 1 September 2025

Available online 2 September 2025

0043-1354/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

challenges: inverse analysis (calibration) and operational optimization (pump operation) in WDN management. Historically, methodologies addressing these tasks have evolved through various water management paradigms (Fu et al., 2024): empirical approaches based on field observations and trial-and-error adjustments; theoretical formulations grounded in classical hydraulic equations such as Hazen–Williams and Darcy–Weisbach (Khedr et al., 2015); computational optimization techniques such as genetic algorithms, gradient-based methods, and differential evolution (Nedaei, 2025; Sarbu, 2021; Sarbu and Popa-Albu, 2023); and more recently, data-centric approaches leveraging machine learning and data-driven modelling (Fu et al., 2022; Meggiorin et al., 2024). Despite their technical maturity, these methods remain difficult for non-experts to interpret and apply, requiring intensive manual tuning and domain expertise that limit real-time usability in operational settings (Hedaiaty Marzouny and Dziedzic, 2024; Ren et al., 2024).

In recent years, Large Language Models (LLMs) have emerged as promising tools for bridging the gap between advanced automation and human interpretability in complex system management (Sami et al., 2024). However, general-purpose LLMs typically generate single-turn responses, and their accuracy and capacity tend to decline as task complexity increases (Zhang et al., 2025b). In contrast, LLM-based agents embed the language model within a structured architecture comprising profile, memory, planning, and action modules, which are implemented through prompt engineering and external mechanisms rather than model retraining (Wang et al., 2024a). The profile module defines the agent's role, personality, and social context through tailored prompts (Masterman et al., 2024; Wang et al., 2024a). The memory module enables short- and long-term recall by injecting retrieved or summarized information into the prompt context as a reminder to avoid model drift (Liu et al., 2025; Wang et al., 2024a). The planning module guides task decomposition and refinement by updating prompts with intermediate reasoning and feedback, where reasoning refers to the LLM's ability to simulate problem solving by breaking down tasks into logical steps (Wang et al., 2024a). The action module converts LLM outputs into executable operations or tool invocations, reintegrating the results into subsequent prompts (Wang et al., 2024a). Together, these components enable autonomous task execution, adaptive feedback integration, and coherent multi-step interaction, transforming LLMs from passive responders to proactive, goal-directed agents.

LLM-based agents are increasingly being explored as central engines in autonomous workflow frameworks across diverse fields. For instance, LLM-based agents have been applied to traffic control systems, where they emulate human-like judgment in managing dynamic, uncertain urban environments (Movahedi and Choi, 2025; Wang et al., 2024b), building energy optimization (Zhang et al., 2025b) and geoscientific data processing (Zhang et al., 2025a). These developments highlight the potential of LLM-based agents beyond language tasks, as their agentic behaviour is supported by iterative dialogue and integration with domain-specific tools. Such innovative use offers a new paradigm for addressing high-level cognitive challenges.

Despite increasing interest in LLM-based agents across domains, their application in the water sector remains limited and largely exploratory. A few studies have demonstrated their potential in constructing domain-specific digital twins and knowledge graphs for water conservation (Yang et al., 2024) and estimating flood depth from imagery (Lyu et al., 2025). Other studies have applied LLMs to water-efficient resource scheduling in data centres (Sami et al., 2024) or to question answering in water engineering (Xu et al., 2025). Within WDN management, only a few studies have emerged: a ChatGPT-assisted framework for pump operation optimization through iterative prompt refinement and EPANET feedback (Hedaiaty Marzouny and Dziedzic, 2024), a multi-agent framework where an orchestrating agent coordinates specialized agents for perception, analytics, modelling, and optimization (Fu, 2025), and an approach exploring the potential of generative AI to support customer interaction, training, and reporting workflows in water utilities (Sela et al., 2025). None of these

efforts systematically evaluates the core capabilities that underpin agentic decision-making in WDN management, including reasoning through prompts, tool calls, executable code generation. Most implementations remain ad hoc, with limited modularity, tool integration, or interaction design, highlighting the need for structured, agentic frameworks to support more autonomous and context-aware decision-making.

To address this gap, we propose and evaluate a novel multi-agent framework in which collaborating LLM-based agents use interactive prompting, simulation tools and autonomous coding to enhance hydraulic decision-making in WDNs. It focuses on two representative optimization tasks: inverse analysis (e.g., hydraulic model calibration) and operational optimization (e.g., pump operation). For each task, an Orchestrating Agent first interprets the task description and the current state of the hydraulic system. This is followed by fixed-round interactions between the Orchestrating Agent and each of three specialized agents, each designed to evaluate a distinct core capability: *reasoning*, where a Knowledge Agent performs formula-guided numerical reasoning through hydraulic principles described in natural language within its system prompt; *simulation tool interaction*, where a Modelling Agent offers physically grounded feedback via simulation tools; and *autonomous coding*, where a Coding Agent executes software code (e.g., Python scripts) generated by the Orchestrating Agent to accomplish specific hydraulic tasks. By evaluating these capabilities on two benchmark networks, i.e., Net2 and Anytown, this study provides a structured, comparative assessment of these agent capabilities in terms of decision quality, computational performance, and adaptability across hydraulic contexts, establishing a foundation for more specialized, tool-integrated LLM agents in water system management.

2. Methodology

This section outlines a multi-agent framework based on LLMs to automate two commonly encountered tasks in WDN management: hydraulic model calibration and pump operation optimization. It details how LLM agents are developed and structured into cohesive planning architecture, emphasizing system design and collaborative agent interactions. Section 2.1 introduces the overall framework used to evaluate the core capabilities of LLM-based agents, while Sections 2.2 and 2.3 present the problem formulations for hydraulic model calibration and pump operation optimization, respectively. Section 2.4 outlines the experimental setting used to implement and evaluate the proposed framework.

2.1. Overview of the LLM-based agent interaction framework

This section presents a multi-agent interaction framework aimed at evaluating the potential of three core capabilities of LLM-based agents in supporting hydraulic decision-making tasks within WDNs. As illustrated in Fig. 1, the framework processes user-defined tasks through three separate modes, each involving a pairwise interaction between an Orchestrating Agent and one specialized agent. Each agent, including the Orchestrating Agent, is defined solely through role alignment and behavioural constraints encoded in its system prompt (see Tables S1–S4 for detailed prompts), which specify its function, domain scope, and response format. The Orchestrating Agent receives a structured task description, interprets the current state of the WDN, and interacts individually with one specialized agent that is explicitly assigned in advance for evaluating a specific capability. For example, a user might submit the following instruction: “Electricity prices are high between 6am and 12pm. Adjust the pump speed to reduce operating costs.” Based on its system prompt, the Orchestrating Agent acts as a hydraulic expert and may respond: “Electricity prices are elevated between 6am and 12pm. I recommend reducing the pump speed to 0.9 during this period. Hydraulic Calculator (Knowledge Agent), please verify the impact through simulation and return the results.” This lightweight design enables the systematic and isolated evaluation of each agent's performance

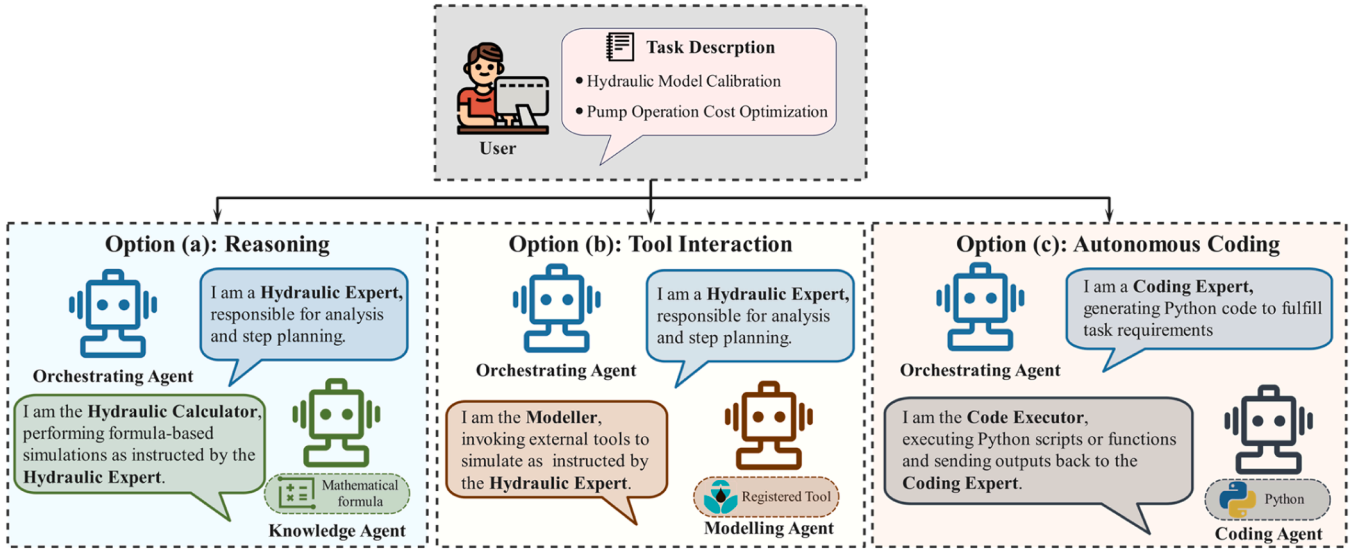


Fig. 1. Overview of the LLM-based Multi-Agent Framework for Automated Solution Development in WDN Management.

across three targeted competencies, which are examined through the following configurations:

- Reasoning Capability**, which is assessed through interaction between the Orchestrating Agent and the Knowledge Agent. In this configuration, the Orchestrating Agent assumes the role of a Hydraulic Expert by analysing the task objectives and network state, planning update strategies, and proposing modifications to parameters such as pipe roughness values or pump speeds. Upon receiving these proposals, the Knowledge Agent performs formula-guided numerical reasoning, which refers to logic-based inference using mathematical formulations such as pressure and head loss equations written in natural language within its system prompt (Tables S1 and S3). Based on these principles, the agent infers the updated WDN state and returns the results to the Orchestrating Agent. This process is repeated iteratively to evaluate the agent's ability to interpret and respond logically to hydraulic scenarios using internalized domain knowledge.
- Simulation Tool Interaction Capability**, which is evaluated via direct interaction between the Orchestrating Agent and the Modelling Agent, with system prompts provided in Tables S1 and S3. This configuration follows a similar mode to Option (a), with the key difference that the Modelling Agent acts solely as a Modeller by calling external simulation tools. Upon receiving instructions from the Orchestrating Agent, it invokes EPANET-based hydraulic simulation functions implemented in Python to perform physically accurate simulations and return model-based feedback. This is enabled through the LLM's native function calling capability, which allows it to interact with registered tools by generating structured input arguments, without relying on free-form code generation (DeepSeek-AI, 2024). This process is used to assess the agent's effectiveness in leveraging external simulations to support and refine decision-making.
- Autonomous Coding Capability**, which is explored through the interaction between the Orchestrating Agent and the Coding Agent, with system prompts provided in Tables S2 and S4. In this setup, the Orchestrating Agent autonomously generates executable Python scripts to address specific hydraulic tasks, with the scripts calling predefined objective functions. These scripts are then executed by the Coding Agent, which returns the outputs to the Orchestrating Agent for analysis and iterative refinement. This setup focuses on evaluating the agent's proficiency in

dynamic code generation and iterative problem-solving through programmatic interaction.

The framework effectively isolates the functional contributions of each agent type and provides insights into both the practical potential and current limitations of LLM-based agents in complex hydraulic decision-making environments.

2.2. Hydraulic model calibration

Calibration constitutes an inverse problem where hydraulic model parameters, such as pipe roughness values, are estimated by minimizing the discrepancy between simulated and observed data. Pipe roughness directly affects head loss and pressure distribution in WDNs, and its accurate calibration is essential for reliable hydraulic modelling (Zhang et al., 2018). Therefore, roughness calibration is selected in this study as a representative task to assess the three core capabilities of LLM-based agents within the proposed framework. The goal is to adjust roughness parameter values so that simulated pressures closely align with observed data. The mean absolute error (MAE) between simulated and observed pressures is used as the objective function:

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{P}_i - P_i| \quad (1)$$

Where $\hat{P}_i$ and P_i represent the simulated and observed pressures at node i , respectively, and N is the total number of monitored nodes.

To perform hydraulic model calibration, the Orchestrating Agent utilizes known system parameters (network topology, observed/initial simulated pressures, nodal demands, and initial MAE values from Table S1) to generate roughness adjustment strategies. These proposed adjustments are then executed by the Knowledge or Modelling Agents, which return simulated hydraulic results to the Orchestrating Agent. However, their computational methodologies differ significantly. The Knowledge Agent performs hydraulic calculations using the Hazen-Williams equation, explicitly described in natural language within its system prompt as the core computational mechanism:

$$Q = k \cdot C \cdot D^{2.63} \cdot \left(\frac{H}{L}\right)^{0.54} \quad (2)$$

where Q is the flow rate (m^3/s), C is the Hazen-Williams roughness coefficient (dimensionless), constrained within a range of 60 to 140 in this

study, D is pipe diameter (m), L is pipe length (m), H is head loss (m), and k is a constant (0.849 in SI units) (Rossman et al., 2020). Using this equation, the Knowledge Agent estimates downstream node pressures based on cumulative head losses from a fixed reservoir head. Further details on the agent's system prompt are provided in Table S1. In contrast, the Modelling Agent uses externally registered hydraulic simulation tools, specifically Python-based EPANET functions, to execute simulations. The pseudocode for these functions is presented in Function S1.

The calibration task handled by the Coding Agent differs notably. Instead of using descriptive text-based input, the Orchestrating Agent receives structured input in the form of file paths (Table S2). Based on these path-based descriptors, the Orchestrating Agent autonomously generates executable Python scripts that read the input files, extract relevant network and pressure data, and apply roughness optimization routines. This automated scripting process emulates the behaviour of professional coders. Subsequently, the Coding Agent executes these scripts locally to complete the calibration tasks.

2.3. Pump operation optimization

Pump stations are the primary consumers of energy in WDNs, with electricity usage reaching up to 90 % in some systems (Hedaiaty Marzouny and Dziedzic, 2024). Enhancing pump operation efficiency can therefore lead to substantial cost savings. To further examine the three capabilities of LLM-based agents, pump operation optimization is selected as a second representative task due to its practical relevance and computational complexity. In this study, the objective is to minimize the total energy cost associated with pump operation over a pre-defined time horizon by adjusting the relative operating speeds of the pumps. The cost is calculated based on the hydraulic head, flow rate, electricity tariff, and pump efficiency, using the following equation:

$$\text{Total Cost} = \frac{\rho g}{3.6 \times 10^6 \cdot \eta} \sum_{t=1}^T (Q_t \cdot H_t \cdot \Delta t \cdot \text{Price}_t) \quad (3)$$

where H_t is the pump head (m), Q_t is the flow rate (m^3/s), Price_t is the electricity price at time t (£/kWh), η is the pump efficiency (dimensionless), ρ is the water density (kg/m^3 , typically 1000) and g is the gravitational acceleration ($9.81 \text{ m}/\text{s}^2$). The energy consumed during each hydraulic time step Δt (s) is obtained by converting hydraulic power (W) into energy (kWh) through division by 3.6×10^6 .

Similar to Section 2.2, in the pump operation optimization task, the Orchestrating Agent uses known task information alongside embedded hydraulic knowledge to propose operational strategies. The Knowledge Agent performs hydraulic simulations based on equations below explicitly provided through system prompts, in addition to Eqs. (2) and (3):

$$\begin{cases} Q_t = Q_{\text{base}} \times s_t \\ H_t = H_{\text{base}} \times s_t^2 \end{cases} \quad (4)$$

Where Q_{base} and H_{base} represent the baseline flow rate and head at full speed, respectively, and s_t is the relative speed of the pump at time t , constrained within a range of 0.85 to 1.15. Further details on the agent's system prompt are provided in Table S3. The Modelling Agent again employs externally registered hydraulic simulation tools using Python-based EPANET functions, with pseudocode detailed in Function S2. The Coding Agent maintains the same structural approach as in the roughness calibration task, where the Orchestrating and Coding Agents generate and execute Python scripts; however, their system prompts are adapted specifically to the pump operation optimization context (Table S4).

2.4. Experimental setting

This section outlines the implementation details and experimental configuration used to evaluate the proposed LLM-based agent framework. The system was developed using the **AutoGen** framework, an open-source platform designed for orchestrating structured communication and task delegation among LLM-based agents (Barbarroxa et al., 2025). **Python** (v3.11) served as the programming environment. All hydraulic simulations were conducted using the Water Network Tool for Resilience (**WNTR**), a Python library that programmatically interfaces with **EPANET 2.2** to support the modelling and analysis of WDNs (Klise et al., 2020). The LLM employed across all agent roles was **DeepSeek-V3** (DeepSeek-AI, 2024) (API deployment, 16 K context window), selected for its demonstrated capability in multi-turn reasoning and Python code generation. To ensure deterministic behaviour and eliminate stochastic variability in outputs, generation parameters were fixed with a temperature of 0, a random seed of 10, a maximum token limit of 2048, and a timeout setting of 600 s.

To reflect the varying complexity of each coordination mode described in Section 2.1, different interaction limits were applied. The Orchestrating Agent was limited to five interaction rounds when engaging with the Knowledge and the Coding Agents, reflecting the relatively lightweight nature of formula-guided numerical reasoning and standalone code execution, as well as preliminary observations that most tasks were completed within three to five turns, which indicates high interaction efficiency. In contrast, up to 30 interaction rounds were permitted during exchanges with the Modelling Agent, as calling external simulation tools hides intermediate reasoning steps, requiring more dialogue to fully demonstrate the agent's decision-making behaviour. All experiments were executed on a fixed computational setup comprising an Intel i7-14700HX CPU and an NVIDIA RTX 4060 GPU (8 GB) to ensure consistency and reproducibility. Agent role prompts, input formats, and task specifications were standardized across all experiments, with full details of system message definitions and initial chat configurations provided in Tables S1–S4 of the Supporting Information.

3. Case study

3.1. Benchmark WDNs

To evaluate the proposed agent-based framework, two benchmark WDNs with distinct topological and operational characteristics were selected, as illustrated in Fig. 2. Net2 is a tree-based network sourced from the EPANET example library, while Anytown is a looped network originally developed as a benchmark for WDN design and operational optimization (Walski et al., 1987). The main structural features of each network are summarized in Table 1, where for the roughness calibration task, the initial roughness value of all pipes was uniformly set to 70 to simulate an uncalibrated baseline condition. The actual pipe roughness values are shown in Fig. S1, where Net 2 mostly consists of pipes with a roughness of 100 and some sections with 70 or 140, while Anytown features roughness values of 70, 120, and 130 across different pipe sections. However, although the pump station location is indicated in Fig. 2, the Net2 lacks explicit pump element definitions (Hoagland et al., 2015), and represents the source and pump station as a single junction node. As a result, pump operation optimization was not performed on Net2, and only hydraulic model calibration was conducted.

3.2. Data preparation

In the proposed agent-based framework, each agent role is defined through natural language system prompts, as detailed in Tables S1–S4. To initiate a task, a task description in natural language format is also required. For the evaluation of *Reasoning* and *Tool Interaction* capabilities, this description includes both the task objective and baseline WDN

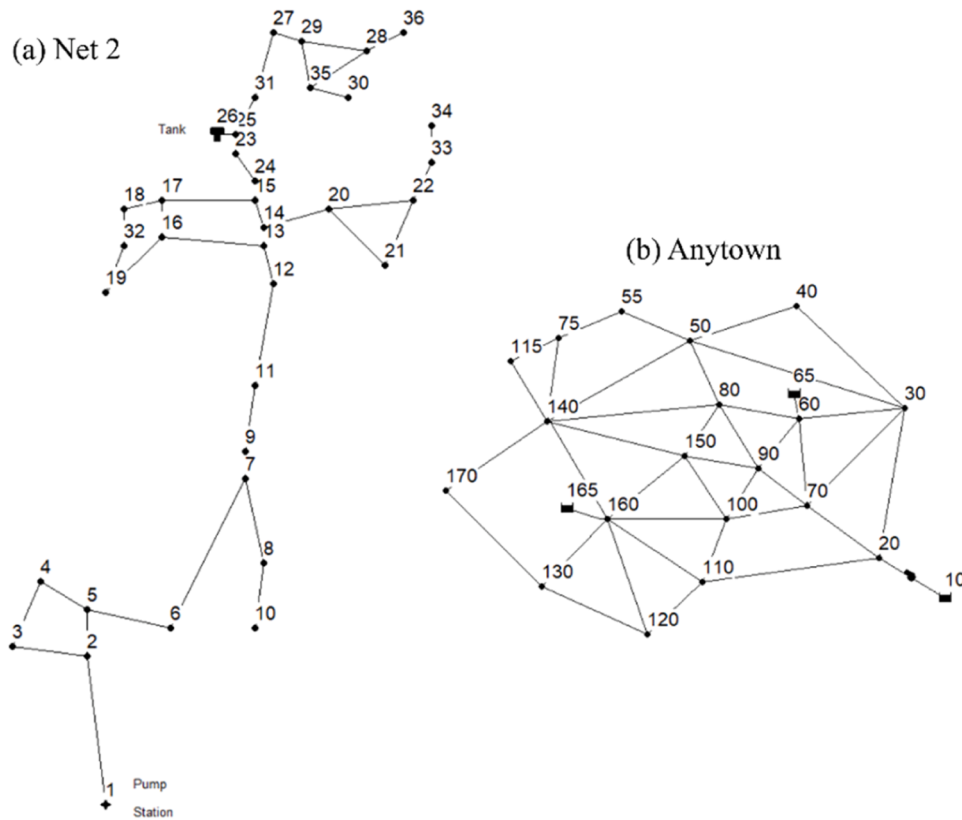


Fig. 2. The topological structure of (a) Net2 and (b) Anytown WDNs.

Table 1
Summary of EPANET-defined structural properties of the selected benchmark WDNs.

WDNs	Reservoirs	Junctions	Tanks	Pipes	Pumps	Initial Pipe Roughness
Net2	None	35	1	40	0	70
Anytown	3	19	None	40	1	70

data as plain text. The inputs typically consist of:

1. WDN Information: The EPANET .inp file content is directly converted into natural language and embedded into the task description.
2. Water Demands: Based on the predefined demand patterns used in the Net2 and Anytown networks [from (Rossman et al., 2020; Walski et al., 1987)], EPANET simulations are run to compute time-varying demands at each node. These demand values are then incorporated into the task as plain-text entries.
3. Observed Values: Using the same water demands as in (2) together with the actual pipe roughness, EPANET simulations are run to extract node pressures over time as observed ground truth values, while predefined electricity prices (£/kWh) for each time interval are also included in the task description.
4. Baseline State: Initial EPANET simulated values, such as pressures from a model with uniform pipe roughness (e.g., roughness = 70, Table 1) or pump energy consumption at each time step under initial settings (e.g., pump speed = 1). These values are likewise embedded into the prompt using natural language.
5. Initial Objective Metric: The baseline error metric (e.g., initial MAE) or cost for comparison.

In contrast, when evaluating the *Autonomous Coding* capability, none of the above datasets are included in the task prompt as plain text.

Instead, the task description provides file paths pointing to the relevant data files. The agents are expected to autonomously generate code to read, process, and utilize these datasets during the calibration or optimization task.

4. Results

4.1. Reasoning capability

This section evaluates the agent's reasoning capability through interactions between the Orchestrating and the Knowledge Agents across three distinct experimental scenarios. In this case, the Knowledge Agent receives update proposals from the Orchestrating Agent and performs computational reasoning based on Eqs. (1) to (4), which are embedded as text in its prompt message. Each experimental scenario was repeated for five times, detailed dialogues and interaction logs of the best-performing rounds recorded in Text Files S1–S3, while Fig. 3 presents a simplified summary of the first-round dialogue, illustrating the human-like reasoning patterns of the orchestrating and knowledge agents, including (1) in Fig. 3, the orchestrating agent identifies nodes with large pressure deviations and updates pipe roughness based on the logic of increasing upstream roughness when simulated pressures are too high, while the knowledge agent performs new simulations and error analysis; and (2) in Fig. 3, the orchestrating agent adjusts pump speeds according to electricity prices, for example by reducing speed during high-price periods and increasing speed during off-peak periods.

Furthermore, to assess the knowledge agent's reasoning accuracy in complex calculations, we re-simulated the optimal roughness and pump speed strategies with EPANET and compared the resulting node pressures and pump costs with the outputs generated by the knowledge agent. Thus, the errors reported in Fig. 4 quantify the numerical deviation between the Knowledge Agent's and EPANET's computed values, for the optimal solution recommended during the agent interaction.

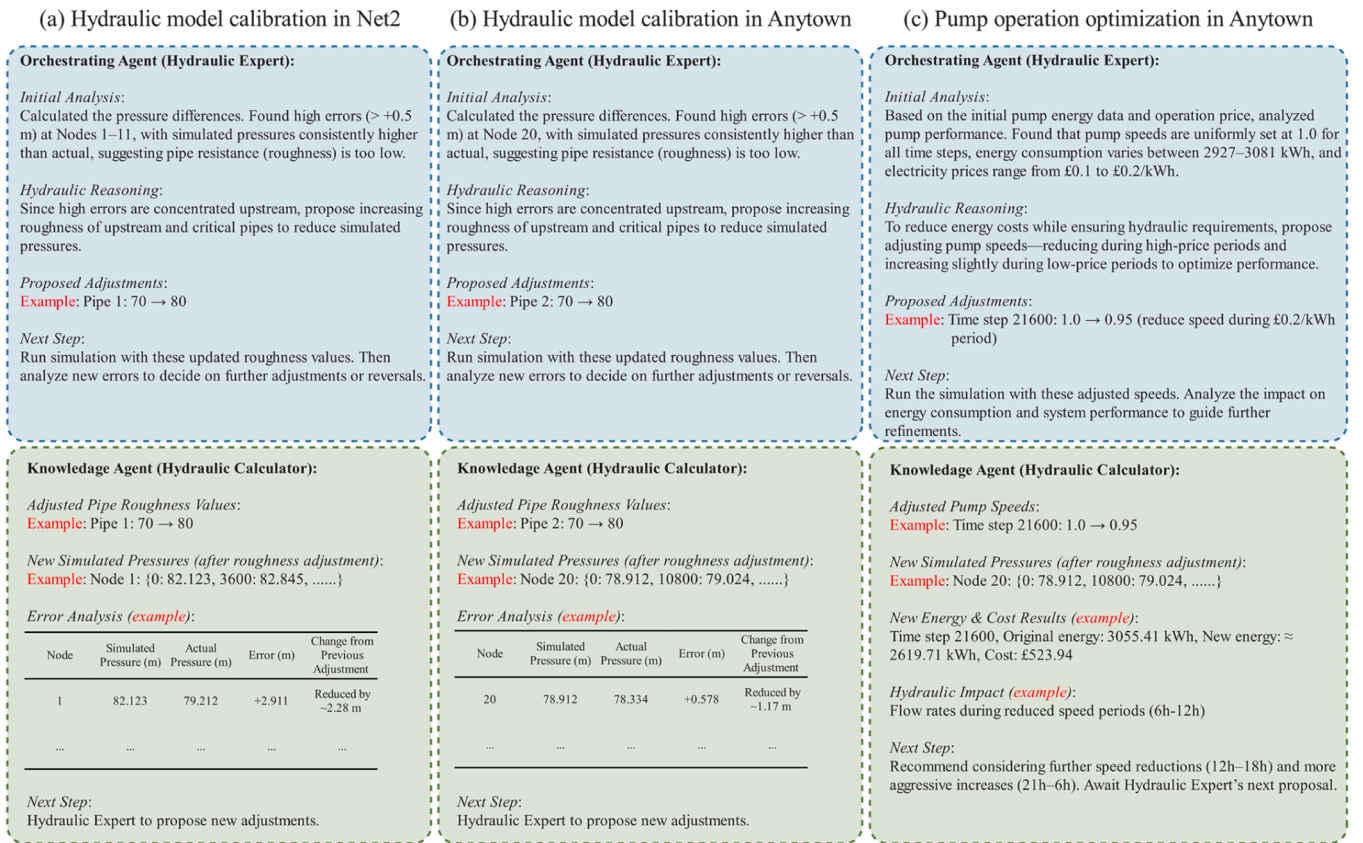


Fig. 3. Simplified First-Round Dialogue Between Orchestrating and Knowledge Agents Across Three Tasks: Hydraulic Calibration in (a) Net2, (b) Anytown, and Pump Operation Optimization in (c) Anytown. The dialogue omits full parameter updates and detailed exchanges, with representative updates highlighted as red examples for illustration.

Fig. 4 illustrates box plot of node pressure error in the Net2 network (first 10 nodes) under final roughness calibration strategy from five experimental repetitions. Notably, the pressure errors at nodes 6 to 10, which are located farther from the source in the branched topology, all exceed 4.00 m. A similar pattern is observed in Fig. 4, but the maximum error occurs at Node 170 and is below 0.60 m, which nevertheless exceeds the 5 % tolerance commonly used in engineering practice (Lingireddy et al., 2004) threshold commonly accepted for engineering applications. Additionally, Fig. 4 shows the pump operation cost error in Anytown over a 24-hour horizon during the pump operation optimization task. The Knowledge Agent exhibits clear errors in cost estimation, particularly between 6 and 18 h, where the maximum discrepancy from EPANET results reaches approximately £150. Although the lower bounds of some box plots reach zero, inspection of Text File S3 reveals that these time steps correspond to periods where pump speeds remained unchanged, requiring no actual reasoning or adjustment by the agent.

Overall, these results indicate that while LLM-based agents are effective in capturing high-level reasoning patterns, they struggle to perform accurate numerical computations using formulas embedded as natural language in their prompt messages. For example, Fig. 4 shows smaller deviations in the looped network despite the expectation that tree-like networks are easier to calibrate, further indicating that the Knowledge Agent alone is insufficient for accurate numerical computation.

4.2. Tool interaction capability

In Section 4.1, the Knowledge Agent was found to be unable to perform accurate hydraulic calculations based on the Hazen-Williams formula described using natural language. Therefore, this section

focuses on the tool-calling capability of the Modelling Agent, which leverages EPANET to perform accurate hydraulic computations. In this case, Modelling Agent interacts with the WDN environment and assess the performance of proposed WDN parameters (roughness values or pump speeds) via predefined Python functions. The same tasks from Section 4.1 were also replicated five times, including pipe roughness calibration for the Net2 and Anytown networks, and pump optimization for the Anytown network. Fig. 5 shows that during nine iterations of hydraulic model calibration and three iterations of pump cost optimization, the MAE and pump cost do not always decrease monotonically, and the final outcomes exhibit noticeable variation across repeated trials. Nevertheless, the results demonstrate the agent's reflective reasoning: the Modelling Agent simulates each updated strategy and returns the outcome, while the Orchestrating Agent evaluates the result against the previous iteration and decides whether further adjustments are needed.

To further analysis this process, full transcripts of the best-performing trials from repeated experiments are available in Supporting Information (Text Files S4–S6), and the corresponding iterative updates are visualized in Fig. 6. In the Net2 calibration task (Fig. 6), the spatial distribution and magnitude of each roughness update are illustrated, showing a 41 % reduction in MAE from 0.652 m to 0.385 m after the ninth iteration. Notably, in iterations 1 to 4, the Orchestrating Agent exhibited expert-like behaviour by first modifying roughness values on major trunk lines and then progressively increasing the number of adjusted pipes. After the fourth iteration, however, the number of modified pipes plateaued at eight. This mirrors typical manual calibration patterns, where modifying too many pipes at once obscures causal relationships and turns reasoning into trial-and-error, increasing the risk of local optima. Although increasing the number of interactions between agents can eventually yield meaningful improvements, escaping local

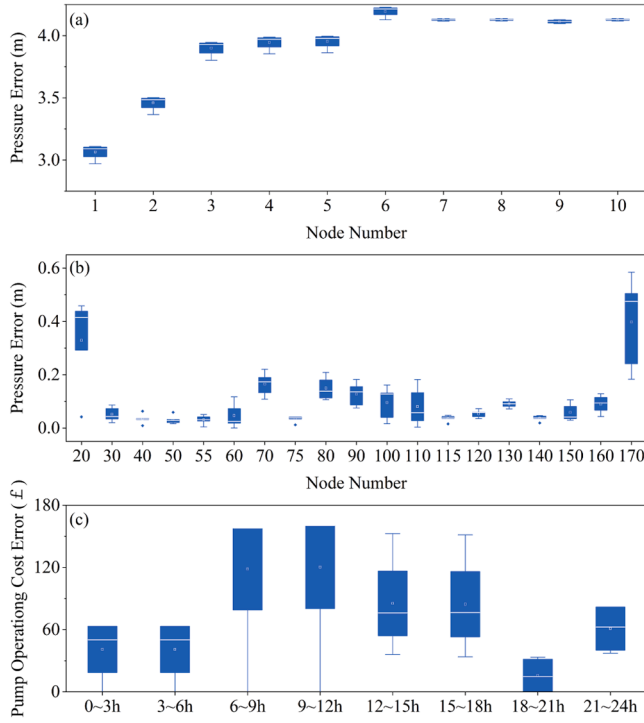


Fig. 4. Comparison of Knowledge Agent and EPANET simulation results in terms of absolute error under the optimal strategy, where (a) and (b) show node pressure comparisons during hydraulic model calibration in Net2 (first 10 nodes) and Anytown respectively, and (c) presents the time-varying pump operation cost in Anytown during pump operation optimization. The box plots were based on five random runs.

optima often incurs a high cost in terms of computational tokens and reasoning cycles. This phenomenon is particularly observable in the Anytown calibration task (Fig. 6), which features a looped network topology. The initial MAE of 0.413 m declined only marginally after the third iteration, with improvements of <0.01 m per iteration, ultimately reaching 0.275 m by the ninth iteration. A closer examination of iterations 5 through 9 reveals that the roughness values assigned to pipes 24 and 56 dropped below the threshold of 60, with similar violations also observed across the other four repeated trials. This violation is not allowed as defined in the natural language prompt in Table S2. This issue was not observed in the simpler, tree-like Net2, but it consistently occurred in the more complex looped Anytown network, where local changes propagate globally and outcomes become harder to predict, even for human experts. As a result, after observing local performance gains, the agent often reinforced prior adjustments while overlooking

constraints, highlighting a key limitation in agent reasoning: prompt accumulation and limited memory retention can lead to model drift, where the agent gradually overlooks predefined constraints during extended dialogue.

For the pump operation optimization task, analysis of Text File S6 reveals the details of each pump speed update, which are summarized in Fig. 7. In Iteration 1, the Orchestrating Agent adjusted the pump speed in line with expert strategies by reducing it to 0.95 from 12 h to 18 h, a peak electricity pricing hour. In Iteration 2, the similar reduced speed of 0.95 was applied earlier from 6 h to 12 h, which led to a lower operational cost of £3375. This behaviour aligns with established domain knowledge that prioritizes load shifting to reduce energy expenditure. While the pump speed was increased to 1.05 during the off-peak hours (0h–6 h and 21h–24 h) in Iterations 1 and 2, the Orchestrating Agent later observed that reducing pump speed generally led to lower costs. As a result, in Iteration 3, the speed during these periods were reduced back to 1.03, further decreasing the total cost to £3236. This change illustrates the agent’s ability to reflect on previous decisions and adjust its strategy based on accumulated experience and outcomes, demonstrating that tool-augmented reasoning by the modelling agent enables more effective optimization compared to the reasoning of the knowledge agent without external simulation tools.

4.3. Autonomous coding capability

In Section 4.2, the Modelling Agent addressed the numerical computation issues observed with the Knowledge Agent in Section 4.1. However, its update strategies still relied on LLM-based human-like reasoning, which in looped networks such as Anytown tended to get trapped in local optima and occasionally violated the constraints imposed by natural language descriptions. Therefore, this section evaluates the Coding Agent, which incorporates autonomous code generation and execution capabilities to address such challenges. Specifically, agents autonomously generate, execute, and refine Python code to solve three representative tasks: hydraulic model calibration for Net2 and Anytown, and pump optimization for Anytown. As in the previous sections, each optimization experiment was repeated five times, and the code logic generated by the Orchestrating Agent across all optimization tasks followed a similar structure. Algorithm 1 summarizes the final optimization logic derived from iterative interactions between the Orchestrating and the Code Agents, capturing the refined structure of the code developed for parameter optimization. The code initializes decision variables x_0 within defined bounds B and defines an error-based objective function $obj(x)$. Without predefined algorithm constraints, the agent autonomously selects suitable optimization routines, often prioritizing methods available in the SciPy library, a widely used Python package for scientific computing, such as gradient-based approaches (e.g., l-BFGS-B) or global search strategies (e.g., Differential Evolution). Across successive iterations, the agent simulates network behaviour,

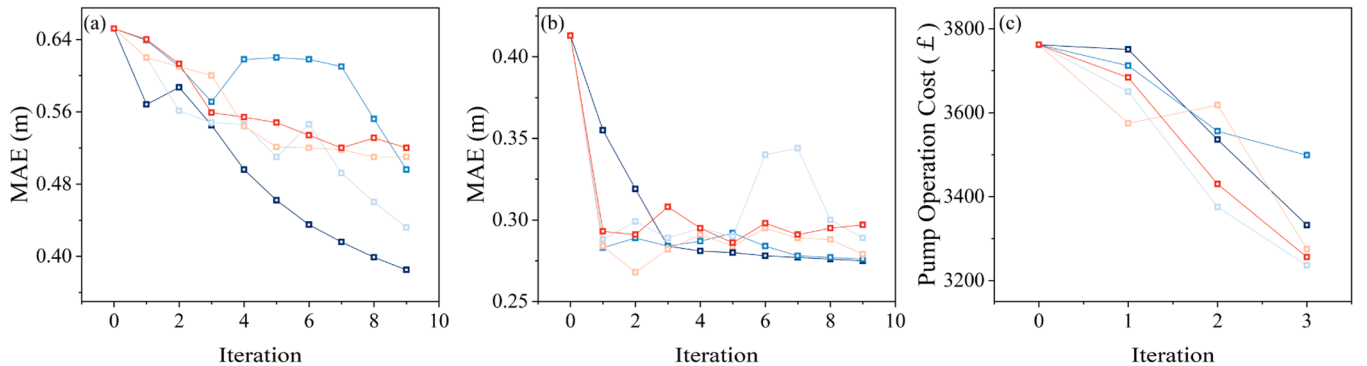


Fig. 5. Convergence of objective functions obtained from the Modelling Agent during interaction with the Orchestrating Agent (five random runs): (a) hydraulic model calibration of Net2, (b) hydraulic model calibration of Anytown, (c) pump optimization of Anytown network.

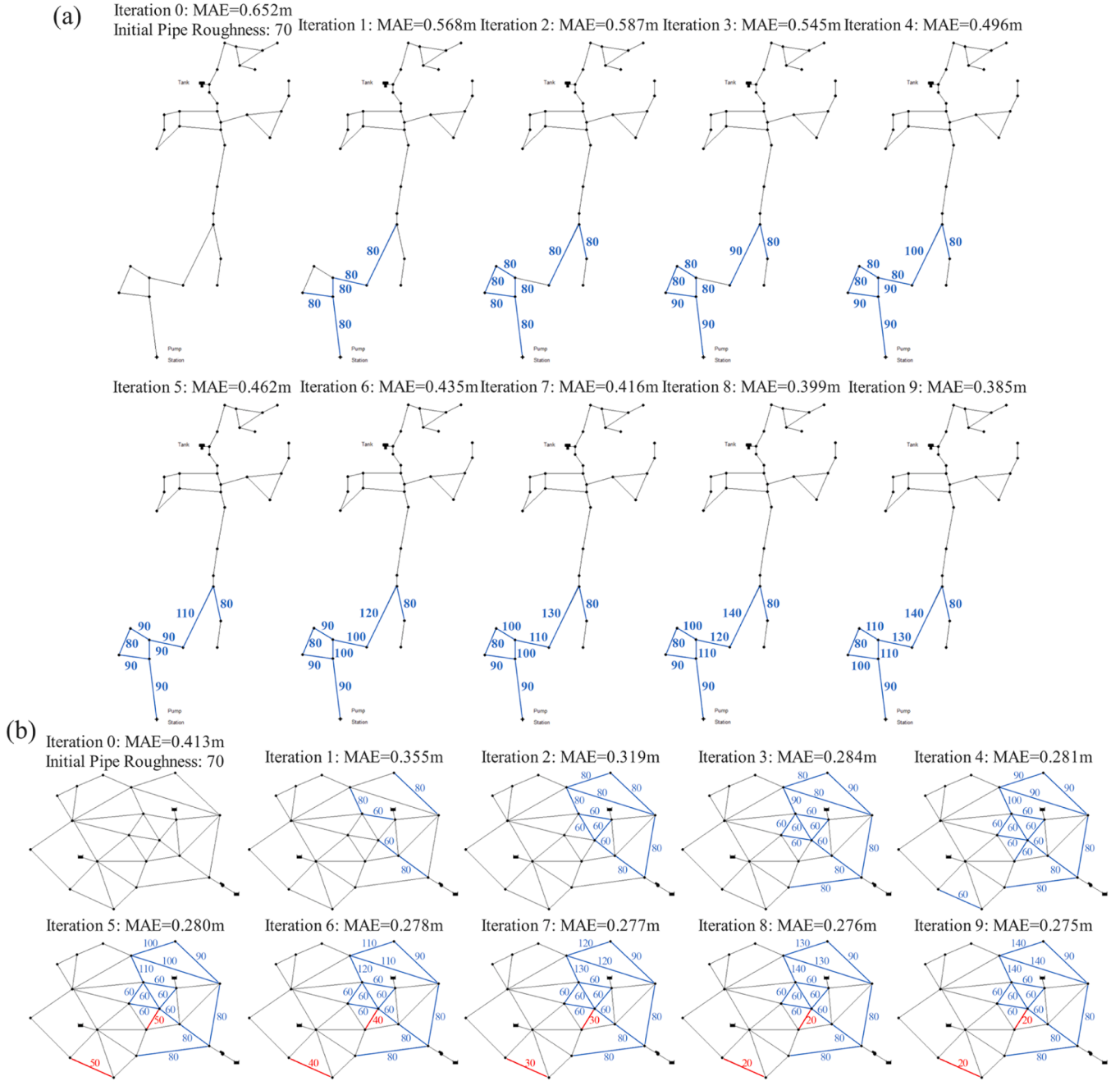


Fig. 6. Spatial distribution and magnitude of pipe roughness updates obtained from the Modelling Agent during interaction with the Orchestrating Agent for hydraulic model calibration in the Net2 (a) and Anytown (b) networks. Blue lines represent updated roughness values within predefined bounds, while red lines indicate values exceeding these bounds.

evaluates candidate solutions, updates parameters, and continues the optimization process until convergence criteria or maximum iterations are reached.

The code shown in Algorithm 1 was autonomously generated through the interaction between the Orchestrating and the Code Agent, without human intervention, and it conforms to standard professional programming practices. Fig. 8 summarizes the execution outcomes of the scripts for three optimization tasks, with each task repeated five times. For the relatively simple Net2 network, the code execution results were consistent across runs, achieving a MAE of 0.0037 m after 50 iterations. In contrast, due to the increased complexity of the Anytown network and the stochastic nature of LLMs, the execution results exhibited some variability. Nevertheless, all runs successfully achieved a MAE of 0.0353 m for hydraulic model calibration and reduced the total

pump operation cost to below £1800. These results significantly outperform those obtained from the Orchestrating and Modelling agents in Fig. 6, respectively, highlighting the agent's ability to transcend the limitations of language-based reasoning by producing precise and verifiable executable code.

Since all groups achieved the optimal objective, three representative runs were randomly selected for analysis, supported by full logs and code in Supporting Information (Text Files S7–S9). A detailed examination of the agent-generated solutions reveals that, in the hydraulic model calibration tasks for Net2 and Anytown (Fig. 9), although the average MAE per node remains below 0.1 m, which significantly outperforms the results shown in Fig. 6, the optimal roughness strategies deviate from the true pipe roughness values by as much as 60 units in the worst-performing pipes, indicating signs of overfitting. In contrast, the

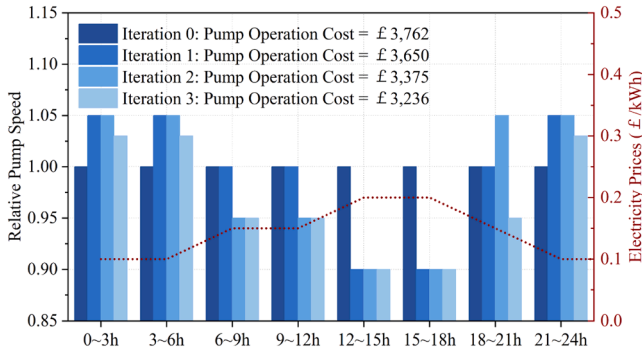


Fig. 7. Variation of relative pump speeds over 24 h and pump operation costs across iterations, obtained from the Modelling Agent during interaction with the Orchestrating Agent, alongside electricity price fluctuations in the Anytown network.

pump operation optimization task in Anytown (Fig. 9) yields pump speeds that remain close to the lower bound of 0.85 throughout the day, yet still exhibit cost-aware behaviour by reducing pump speed during the peak electricity pricing period and shifting loads to off-peak hours. While part of the discrepancy can be attributed to calibration data overfitting, the findings suggest that code generation alone may not always produce robust or practical solutions. A more reliable approach may involve combining code generation with agent-based reasoning and external simulation tools to enhance performance in complex WDN tasks.

5. Discussion

5.1. Token consumption and the central role of LLMs in agent performance

In this study, since each agent interaction involved multiple rounds of prompts and responses from LLMs, token consumption was adopted as the primary metric to quantify the computational cost of agentic reasoning, with results shown in Table 2. A token refers to the basic unit of information processed during language understanding and generation (DeepSeek-AI, 2024), and the number of tokens directly affects both computational cost and processing speed, with more tokens leading to higher costs and slower performance. The highest token usage was observed in interactions between the Orchestrating Agent and the Knowledge Agent, particularly in the Hydraulic Model Calibration (Net2) task, where up to 265,694 tokens were consumed. This was

mainly due to the complexity and length of the Knowledge Agent's responses, which encoded strategic reasoning and optimization logic. Even after 30 rounds of interaction with the Modelling Agent, token consumption remained low because tool operations were executed through local functions without incurring additional token costs. The lowest token usage was found in interactions with the Coding Agent, typically below 100,000 tokens. This was achieved by passing structured data, such as network parameters as variables rather than being expressed in natural language, which would otherwise require more tokens. However, as analysed in Section 4, invoking each agent's capability individually led to different failure problems when addressing complex tasks. This suggests that while higher token usage reflects more complex reasoning, it does not guarantee success. Future frameworks should balance agent capabilities and token efficiency to improve overall performance.

In addition, the importance of LLM quality within the agentic framework was assessed through a controlled experiment in which a weaker model, DeepSeek-1.16B, was substituted for DeepSeek-V3 in the hydraulic calibration task on the Anytown network (see Supplementary Text File S10). Despite using identical prompts and system message configurations, the agent repeatedly failed to invoke the simulation tool correctly, generating hallucinated input parameters that failed to match the expected format required by the external tool. This failure highlights the inability of earlier models to support precise input formatting and forward planning, ultimately preventing the agent from completing the reasoning loop.

This example shows the importance of using a robust and well-aligned LLM as the foundation of the agent framework. While general-purpose models like DeepSeek-V3 performed well in this study, they may still be insufficient for domain-specific, numerically sensitive tasks such as hydraulic optimization. Future work should focus on developing specialized models like WaterGPT (Ren et al., 2024), trained specifically for water systems and enhanced with retrieval-augmented generation (RAG) techniques (Xu et al., 2024) to improve agent accuracy, reliability, and adaptability in complex operational scenarios.

5.2. Prompt, multi-agency and workflow

Beyond the capabilities of the underlying LLM discussed in Section 5.1, the performance of agent-based frameworks also depends heavily on prompt clarity and workflow. Effective delegation requires task descriptions to be precise and behaviourally aligned (Guo et al., 2024; Kolt, 2025; Lyu et al., 2025; Wang et al., 2024a). In this study, we demonstrate the impact of prompt quality through a controlled experiment using a reasoning agent with external simulation tools for roughness

Algorithm 1

Parameter Optimization for Water Distribution Networks.

Input: Water distribution network model W , simulation function $F(x, W)$, observed data D_{obs} (pressure or energy cost), optimization bounds B , maximum iterations max_iter .
Output: Optimal parameters x^* , simulation results $F(x^*, W)$, evaluation metric $E(x^*)$

- 1: Load water distribution network model W .
- 2: Load observed data D_{obs} .
- 3: Initialize decision variables x_0 within bounds B .
- 4: Define the objective function:
 $obj(x) = Error(F(x, W), D_{obs})$
- 5: Configure optimization algorithm:
Randomly select an optimizer from the Python library `scipy.optimize`
- 6: Set iteration counter $t \leftarrow 0$.
- 7: Set best error $E_{best} \leftarrow \infty$.
- 8: Repeat until $t \geq max_iter$ or convergence criteria met:
Generate candidate solution x_t within bounds B .
Simulate network output: $S_t \leftarrow F(x_t, W)$.
Compute current error: $E_t \leftarrow obj(x_t)$.
If $E_t < E_{best}$, then:
Update best solution: $x^* \leftarrow x_t$, $E_{best} \leftarrow E_t$.
Increment iteration counter: $t \leftarrow t + 1$.
- 9: Evaluate the optimal solution $F(x^*, W)$.
- 10: **Return** optimal parameters x^* , simulation output $F(x^*, W)$, evaluation metric $E(x^*)$.

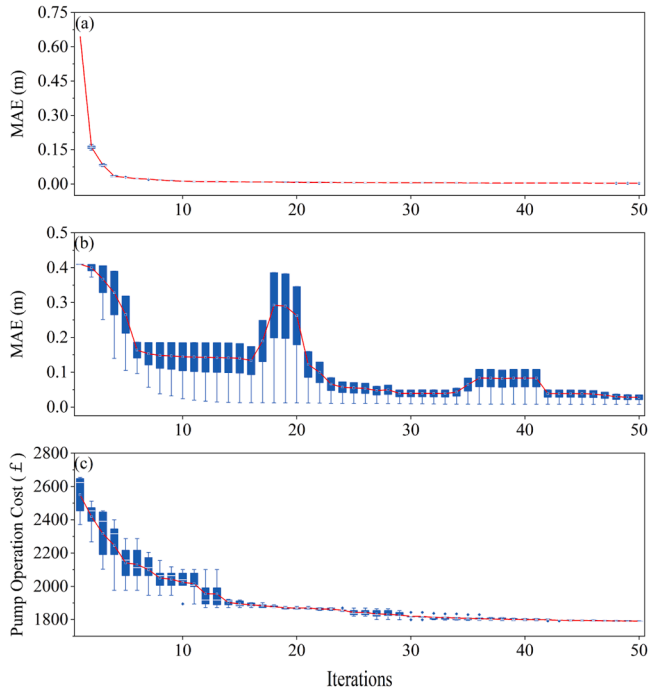


Fig. 8. Convergence of objective functions for Coding Agents across three optimization tasks: (a) and (b) represent hydraulic model calibration for Net2 and Anytown, respectively; (c) shows pump operation cost optimization in Anytown. The box plots were based on five random runs.

calibration in the Anytown network. When the “Updated Analysis Logic” section was removed from the Orchestrating agent’s system message [Table S1], the agent lacked guidance on when and how to update its decisions. As shown in Fig. 10, this led to repetitive suboptimal outputs, with MAE values fluctuating between 0.289 m and 0.312 m. This highlights the importance of prompts that are not only goal-directed but also structured to reflect expert reasoning and staged decision logic.

Our study employs a collaborative dialogue framework comprising an orchestrating agent and specialized agents, which leverages the underlying LLM’s capabilities for iterative reasoning, tool invocation, and autonomous code generation to accomplish hydraulic model calibration and pump operation optimization. However, for more complex and fine-grained tasks, each sub-task might require a specialized agent, but this does not imply simply increasing the number of agents indiscriminately. Recent findings by (Tian and Zhang, 2024) emphasize that merely adding more agents does not inherently improve system performance. Instead, the effectiveness of multi-agent systems fundamentally depends on meticulous role delineation and clearly defined coordination

mechanisms among agents.

As the number of agents increases, clearly defining and understanding each phase of the current task becomes increasingly critical for managing complexity and maintaining consistency in reasoning. Although our study specifically employs an evaluator-optimizer workflow, there exist several alternative workflow designs suitable for handling more refined and complex tasks. These workflows include: (1) a chain-of-thought workflow, sequentially organizing multiple steps with each step’s output serving as input for the next step (Ferrag et al., 2025); (2) a parallel pipeline workflow, where multiple agents concurrently execute separate sub-tasks and integrate their outputs subsequently (Masterman et al., 2024); and (3) a conditional routing workflow, dynamically adjusting the execution path based on intermediate results and predefined conditions (Yue et al., 2025). As each workflow strategy entails distinct trade-offs in scalability, flexibility, and coordination cost, selecting an architecture aligned with task characteristics will be critical for enabling LLM-based agents to effectively support automated water management.

5.3. Findings, limitations and future work

This study presents an analysis of the application of LLM-based agents in the management of WDNs. The findings demonstrate that such agents are capable of interpreting domain-specific information and exhibiting human-like reasoning and reflection. The Knowledge Agent performs well in logical reasoning but cannot achieve accurate numerical computation when relying solely on formulas expressed using natural language. When integrated with external simulation tools, the Modelling Agent interacts effectively with the hydraulic environment and provides physics-consistent feedback for precise hydraulic calculations, although its update strategies sometimes converge to sub-optimal solutions in looped networks. Finally, interaction with the Coding Agent enables the autonomous generation of executable code with high computational precision to address hydraulic tasks and achieve near-optimal results. On the whole, these capabilities position LLM-based

Table 2
Average token consumption per task.

	Hydraulic Model Calibration (Net2)	Hydraulic Model Calibration (Anytown)	Pump Operation Cost Optimization (Anytown)
Orchestrating ↔ Knowledge Agent	265,694	179,172	123,282
Orchestrating ↔ Modelling Agent	169,248	195,320	157,055
Orchestrating ↔ Coding Agent	60,586	53,586	51,964

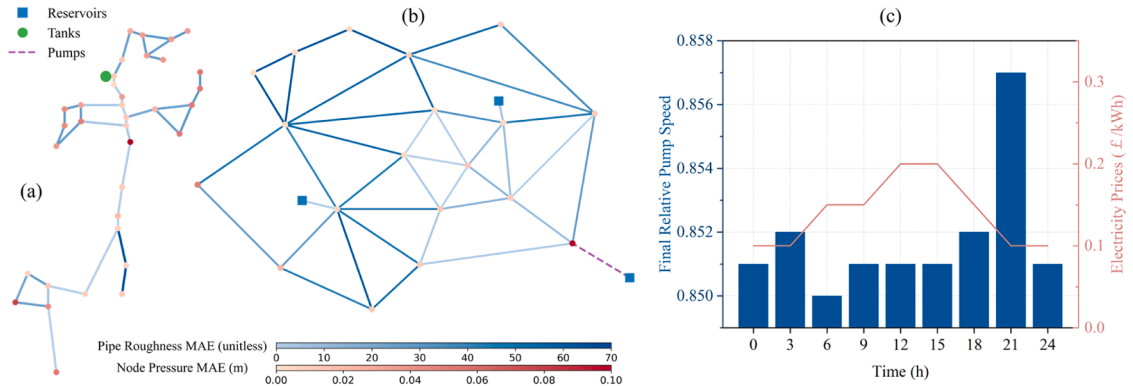


Fig. 9. Optimal solutions from code-generated optimization: (a) and (b) show the absolute roughness error between optimized and true values for hydraulic model calibration in Net2 and Anytown, respectively; (c) presents the optimal pump strategy.

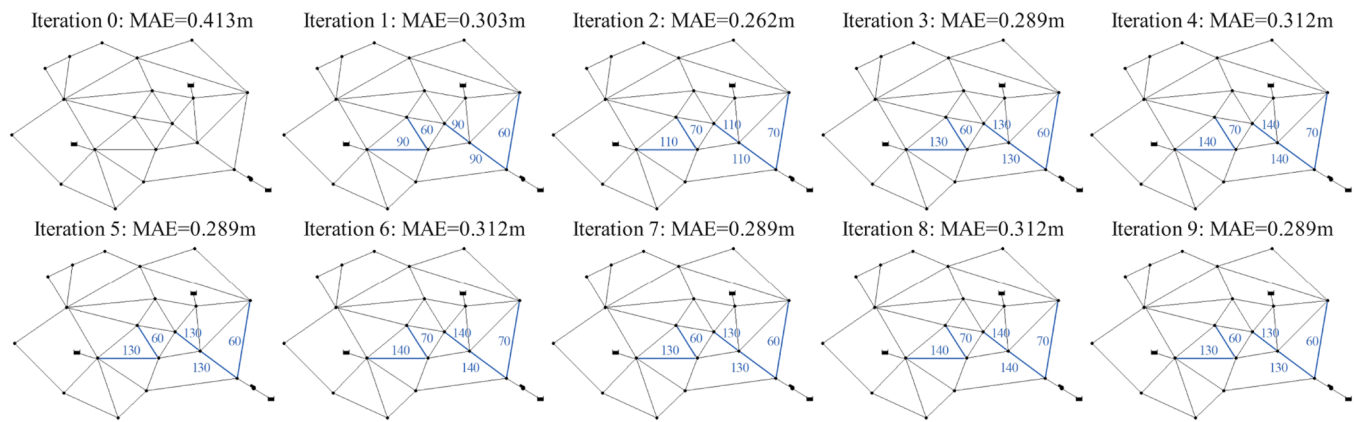


Fig. 10. Pipe roughness updates during hydraulic model calibration in Anytown, based on interactions between the Orchestrating and Modelling Agents in the absence of explicit update instructions in the prompt.

agents as promising tools for replicating and automating many traditionally manual decision-making processes in WDN operations.

Beyond these findings, several limitations of this study must be acknowledged. First, the case studies were limited to relatively simple and clearly bounded tasks. Scaling LLM-based agents to more complex and larger WDNs involving interdependent components or real-time adaptive control strategies remains an open challenge. Second, the reasoning-only mode required the encoding of entire network structures and hydraulic parameters in plain text format, which could easily exceed the input capacity of current LLMs and negatively impact memory efficiency and reasoning coherence. This suggests future extensions to leverage code or tool-based variable passing instead of text. Third, this study adopted a fully autonomous agent workflow, where only initial task definitions were provided by humans, suggesting that incorporating periodic human input could improve alignment with operational goals and adaptability to dynamic scenarios.

To address these limitations, future research will explore hybrid multi-agent frameworks that combine reasoning, tool integration, and autonomous coding within a unified architecture (Chen et al., 2025). Such systems could dynamically select the most appropriate capabilities based on task complexity and data availability, enabling more flexible and robust performance across diverse operational contexts. Additionally, the development of domain-specialized LLMs, combined with RAG to enhance knowledge grounding (Xu et al., 2024), is critical for scaling agentic frameworks to complex, real-time water management scenarios (Ren et al., 2024). Building on this first step in demonstrating the potential of AI agents for hydraulic decision-making, future work should also include systematic performance comparisons with expert-developed methods to better assess and enhance their capabilities. These advancements may ultimately enable human operators to manage complex water systems through a single dialogue box by automating decisions via natural language interaction.

6. Conclusions

This study systematically evaluated the capabilities and limitations of LLM-based agents in managing WDN optimization tasks, exemplified by inverse analysis (hydraulic model calibration) and operational optimization (pump optimization). An Orchestrating Agent was used to interact with three types of agents: the Knowledge Agent, the Modelling Agent, and the Coding Agent. These interactions were designed to explore the agents' respective strengths in reasoning, external tool integration, and autonomous code generation and execution. Based on this study, the main conclusions are as follows:

1. The LLM-based agents demonstrate strong reasoning and reflective abilities, enabling human-like decision-making, but embedding

formulas as natural language in the prompt does not allow them to perform complex hydraulic calculations accurately.

2. Agents interacting with the environment via external simulation tools exhibit improved accuracy and reliability. Nevertheless, the use of natural language to define numerical constraints presents challenges, as such "soft" constraints are not always strictly enforced, and agents may exhibit tendencies toward local optima, particularly in looped networks.
3. Autonomous code generation by agents significantly improved optimization performance and computational efficiency. While this approach overcame the limitations of reasoning-based methods through precise execution, its effectiveness, like many algorithmic methods, remains dependent on the input data.

Overall, this study highlights the promising potential of LLM-based agents for WDN management. These agents offer a novel paradigm by enabling goal-driven, natural language-based interaction and task execution. However, their application remains in the early exploration phase, particularly regarding their capability to address domain-specific, numerically intensive problems. Future work should focus on the development of specialized, domain-adapted LLMs that can integrate multi-tool workflows and combine language-based reasoning with code generation to improve applicability and effectiveness in complex hydraulic scenarios.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT 4o in order to improve language and readability. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

CRediT authorship contribution statement

Jian Wang: Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis. **Guangtao Fu:** Writing – review & editing, Supervision, Funding acquisition. **Dragan Savic:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Jian Wang is a PhD student sponsored by the China Scholarship Council (202408340010) at the University of Exeter. Dragan Savic has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (grant agreement no [951424]). Guangtao Fu has received funding from the British Council under the UK-China Institutional Partnership. For the purpose of open access, the author has applied a Creative Commons Attribution (CC BY) licence to any Author Accepted Manuscript version arising from this submission.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.watres.2025.124536](https://doi.org/10.1016/j.watres.2025.124536).

Data availability

The code supporting this study is available at https://github.com/wangjian169/LLMs_for_Automating_WDNs_Optimization.

References

- Barbarroxa, R., Gomes, L., Vale, Z., 2025. Benchmarking large language models for multi-agent systems: a comparative analysis of AutoGen, CrewAI, and TaskWeaver. In: Mathieu, P., De la Prieta, F. (Eds.), *Advances in Practical Applications of Agents, Multi-Agent Systems, and Digital Twins: The PAAMS Collection*. Springer Nature Switzerland, Cham, pp. 39–48. https://doi.org/10.1007/978-3-031-70415-4_4.
- Batista do Ego, T., de Azevedo, J.R.G., Marques Bezerra, S., de, T., 2023. Optimization of the operation of water distribution systems with emphasis on the joint optimization of pumps and reservoirs. *Water Supply* 23, 1094–1105. <https://doi.org/10.2166/ws.2023.065>.
- Chen, Y., Jhamtani, H., Sharma, S., Fan, C., Wang, C., 2025. Steering large language models between code execution and textual reasoning. <https://doi.org/10.48550/arXiv.2410.03524>.
- DeepSeek-AI, 2024. DeepSeek-V3 technical report (Technical Report No. [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)). ArXiv.
- Ferrag, M.A., Tihanyi, N., Debbah, M., 2025. Reasoning beyond limits: advances and open problems for LLMs. <https://doi.org/10.48550/arXiv.2503.22732>.
- Fu, G., 2025. Towards autonomous planning and management of urban water systems. *ESS Open Archive*. <https://doi.org/10.22541/essoar.175745445.50927919.v1>.
- Fu, G., Jin, Y., Sun, S., Yuan, Z., Butler, D., 2022. The role of deep learning in urban water management: a critical review. *Water Res.* 223, 118973. <https://doi.org/10.1016/j.watres.2022.118973>.
- Fu, G., Savic, D., Butler, D., 2024. Making waves: towards data-centric water engineering. *Water Res.* 256, 121585. <https://doi.org/10.1016/j.watres.2024.121585>.
- Gong, J., Guo, X., Yan, X., Hu, C., 2023. Review of urban drinking water contamination source identification methods. *Energies* 16, 705. <https://doi.org/10.3390/en16020705>.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X., 2024. Large language model based Multi-agents: a survey of progress and challenges. <https://doi.org/10.48550/arXiv.2402.01680>.
- Hedaiaty Marzouny, N., Dziedzic, R., 2024. AI-assisted pump operation for energy-efficient water distribution systems. In: *The 3rd International Joint Conference on Water Distribution Systems Analysis & Computing and Control for the Water Industry (WDSA/CCWI 2024)*. Presented at the International Joint Conference on Water Distribution Systems Analysis & Computing and Control for the Water Industry. MDPI, p. 3. <https://doi.org/10.3390/engproc2024069003>.
- Hoagland, S., Schal, S., Ormsbee, L., Bryson, S., 2015. Classification of water distribution systems for research applications. In: *World Environmental and Water Resources Congress 2015*. Presented at the World Environmental and Water Resources Congress 2015. Austin, TX. American Society of Civil Engineers, pp. 696–702. <https://doi.org/10.1061/9780784479162.064>.
- Khedr, A., Tolson, B., Ziemann, S., 2015. Water distribution system calibration: manual versus optimization-based approach. *Proced. Eng.* 119, 725–733. <https://doi.org/10.1016/j.proeng.2015.08.926>.
- Klise, K., Hart, D., Bynum, M., Hogge, J., Haxton, T., Murray, R., Burkhardt, J., 2020. *Water Network Tool For Resilience (WNTR) User Manual: Version 0.2.3* (No. EPA/600/R-20/185). U.S. EPA Office of Research and Development, Washington, DC.
- Kolt, N., 2025. Governing AI agents. <https://doi.org/10.48550/arXiv.2501.07913>.
- Lingireddy, S., Ormsbee, L.E., Wood, D.J., 2004. Calibration of hydraulic network models. In: Lehr, J.H., Keeley, J. (Eds.), *Water Encyclopedia*. Wiley, pp. 313–320. <https://doi.org/10.1002/047147844X.sw71>.
- Liu, B., Li, X., Zhang, J., Wang, J., He, T., Hong, S., Liu, H., Zhang, S., Song, K., Zhu, K., Cheng, Y., Wang, S., Wang, X., Luo, Y., Jin, H., Zhang, P., Liu, O., Chen, J., Zhang, H., Yu, Z., Shi, H., Li, B., Wu, D., Teng, F., Jia, X., Xu, J., Xiang, J., Lin, Y., Liu, Tianming, Liu, Tongliang, Su, Y., Sun, H., Berseht, G., Nie, J., Foster, I., Ward, L., Wu, Q., Gu, Y., Zhuge, M., Tang, X., Wang, H., You, J., Wang, C., Pei, J., Yang, Q., Qi, X., Wu, C., 2025. Advances and challenges in foundation agents: from brain-inspired intelligence to evolutionary, collaborative, and safe systems. <https://doi.org/10.48550/arXiv.2504.01990>.
- Lyu, H., Zhou, S., Wang, Z., Fu, G., Zhang, C., 2025. Assessing large multimodal models for urban floodwater depth estimation. *Water Resour. Res.* 61, e2024WR039494. <https://doi.org/10.1029/2024WR039494>.
- Makaremi, Y., Haghighi, A., Ghafouri, H.R., 2017. Optimization of pump scheduling program in water supply systems using a self-adaptive NSGA-II: a review of theory to real application. *Water Resour. Manage.* 31, 1283–1304. <https://doi.org/10.1007/s11269-017-1577-x>.
- Masterman, T., Besen, S., Sawtell, M., Chao, A., 2024. The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: a survey. <https://doi.org/10.48550/arXiv.2404.11584>.
- Meggiorin, M., Naranjo-Fernández, N., Passadore, G., Sottani, A., Botter, G., Rinaldo, A., 2024. Data-driven statistical optimization of a groundwater monitoring network. *J. Hydrol.* 631, 130667. <https://doi.org/10.1016/j.jhydrol.2024.130667>.
- Movahedi, M., Choi, J., 2025. The crossroads of LLM and traffic control: a study on large language models in adaptive traffic signal control. *IEEE Transact. Intell. Transport. Syst.* 26, 1701–1716. <https://doi.org/10.1109/TITS.2024.3498735>.
- Nedaei, M., 2025. Impact of network configuration on hydraulic constraints and cost in the optimization of water distribution networks. *Appl. Sci.* 15, 3126. <https://doi.org/10.3390/app15063126>.
- Ren, Y., Zhang, T., Dong, X., Li, W., Wang, Z., He, J., Zhang, H., Jiao, L., 2024. WaterGPT: training a large language model to become a hydrology expert. *Water (Basel)* 16, 3075. <https://doi.org/10.3390/w16213075>.
- Rossman, L., Woo, H., Tryby, M., Shang, F., Janke, R., Haxton, T., 2020. *EPANET 2.2 User Manual*. U.S. Environmental Protection Agency, Washington, DC.
- Sami, A.M., Rasheed, Z., Kemell, K.-K., Waseem, M., Kilamo, T., Saari, M., Duc, A.N., Systä, K., Abrahamsson, P., 2024. System for systematic literature review using multiple AI agents: concept and an empirical evaluation. <https://doi.org/10.48550/arXiv.2403.08399>.
- Sarbu, I., 2021. Optimization of Urban water distribution networks using deterministic and heuristic techniques: comprehensive review. *J. Pipeline Syst. Eng. Pract.* 12, 03121001. [https://doi.org/10.1061/\(ASCE\)PS.1949-1204.0000575](https://doi.org/10.1061/(ASCE)PS.1949-1204.0000575).
- Sarbu, I., Popa-Albu, S., 2023. Optimization of urban water distribution networks using heuristic methods: an overview. *Water Int.* 48, 120–148. <https://doi.org/10.1080/02508060.2022.2127611>.
- Sela, L., Sowby, R.B., Salomons, E., Housh, M., 2025. Making waves: the potential of generative AI in water utility operations. *Water Res.* 272, 122935. <https://doi.org/10.1016/j.watres.2024.122935>.
- Sharif, M.N., Bakhtavar, E., Haider, H., Hu, G., Hewage, K., Sadiq, R., 2022. Staged energy and water quality optimization for large water distribution systems. *Environ. Monit. Assess.* 194, 232. <https://doi.org/10.1007/s10661-022-09874-0>.
- Tian, C., Zhang, Y., 2024. Optimizing collaboration of LLM based agents for finite element analysis. <https://doi.org/10.48550/arXiv.2408.13406>.
- Walski, T.M., Brill, E.D., Gessler, J., Goulter, I.C., Jeppson, R.M., Lansey, K., Lee, H., Liebman, J.C., Mays, L., Morgan, D.R., Ormsbee, L., 1987. Battle of the Network Models: epilogue. *J. Water Resour. Plann. Manage.* 113, 191–203. [https://doi.org/10.1061/\(ASCE\)0733-9496\(1987\)113:2\(191\)](https://doi.org/10.1061/(ASCE)0733-9496(1987)113:2(191)).
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., Wen, J.-R., 2024a. A survey on large language model based autonomous agents. *Front. Comput. Sci.* 18, 186345. <https://doi.org/10.1007/s11704-024-40231-1>.
- Wang, M., Pang, A., Kan, Y., Pun, M.-O., Chen, C.S., Huang, B., 2024b. LLM-assisted light: leveraging large language model capabilities for Human-mimetic traffic signal control in complex urban environments. <https://doi.org/10.48550/arXiv.2403.08337>.
- Xu, B., Li, Z., Yang, Yuxin, Wu, G., Wang, C., Tang, X., Li, Y., Wu, Z., Su, Q., Shi, X., Yang, Yue, Tong, R., Wen, L., Ng, H.Y., 2025. Evaluating and advancing large language models for water knowledge tasks in engineering and research. *Environ. Sci. Technol. Lett.* 12, 289–296. <https://doi.org/10.1021/acs.estlett.5c00038>.
- Xu, H., Yuan, J., Zhou, A., Xu, G., Li, W., Ban, X., Ye, X., 2024. GenAI-powered Multi-agent paradigm for Smart Urban mobility: opportunities and challenges for integrating large language models (LLMs) and retrieval-augmented generation (RAG) with intelligent transportation systems. <https://doi.org/10.48550/arXiv.2409.00494>.
- Yang, Y., Ye, F., Xu, D., Zhang, X., Xu, J., 2024. LLM-based Digital Twin Water Conservancy Knowledge Graph construction. In: *2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI)*. Presented at the 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI), pp. 656–662. <https://doi.org/10.1109/ICTAI62512.2024.00098>.
- Yue, Y., Zhang, G., Liu, B., Wan, G., Wang, K., Cheng, D., Qi, Y., 2025. MasRouter: learning to route LLMs for Multi-agent systems. <https://doi.org/10.48550/arXiv.2502.11133>.
- Zaman, D., Tiwari, M.K., Gupta, A.K., Sen, D., 2021. Performance indicators-based energy sustainability in urban water distribution networks: a state-of-art review and conceptual framework. *Sustain. Citi. Soc.* 72, 103036. <https://doi.org/10.1016/j.scs.2021.103036>.
- Zarei, N., Azari, A., Heidari, M.M., 2022. Improvement of the performance of NSGA-II and MOPSO algorithms in multi-objective optimization of urban water distribution networks based on modification of decision space. *Appl. Water Sci.* 12, 133. <https://doi.org/10.1007/s13201-022-01610-w>.

- Zhang, J., Clairmont, C., Que, X., Li, W., Chen, W., Li, C., Ma, X., 2025a. Streamlining geoscience data analysis with an LLM-driven workflow. *Appl. Comput. Geosci.* 25, 100218. <https://doi.org/10.1016/j.acags.2024.100218>.
- Zhang, L., Ford, V., Chen, Z., Chen, J., 2025b. Automatic building energy model development and debugging using large language models agentic workflow. *Energy Build.* 327, 115116. <https://doi.org/10.1016/j.enbuild.2024.115116>.
- Zhang, Q., Zheng, F., Duan, H.-F., Jia, Y., Zhang, T., Guo, X., 2018. Efficient numerical approach for simultaneous calibration of pipe roughness coefficients and nodal demands for water distribution systems. *J. Water Resour. Plann. Manage.* 144, 04018063. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0000986](https://doi.org/10.1061/(ASCE)WR.1943-5452.0000986).